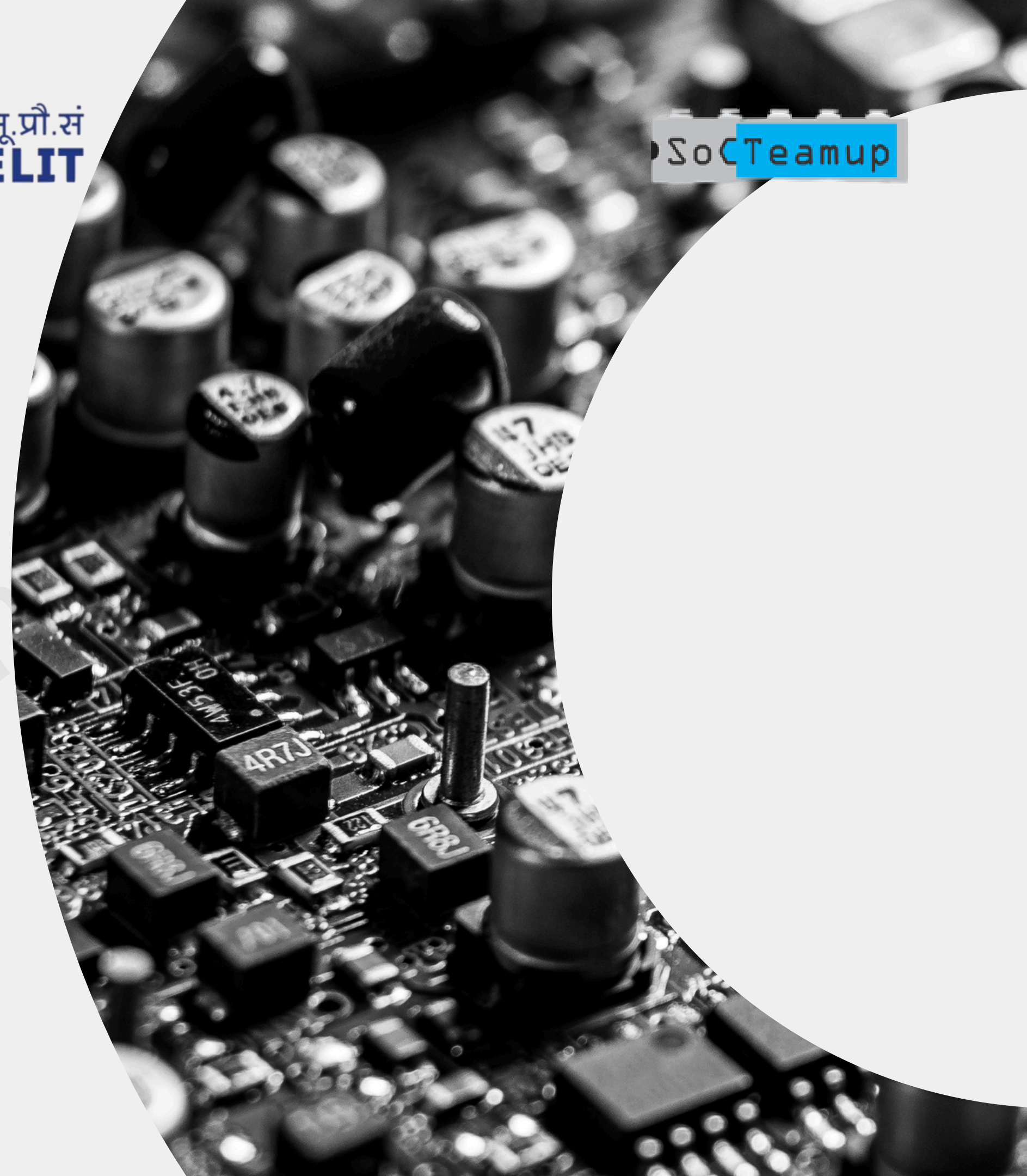


Verilog Modeling Fundamentals

etchip





Outline

Topic
Introduction to Verilog
Verilog Syntax Basics
Module Structure
Design Hierarchy
Modeling Styles in Verilog
Behavioral Modeling (Concept)
Behavioral Modeling (Use Cases)
Dataflow Modeling (Concept)
Dataflow Modeling (Use Cases)
Structural Modeling (Concept)
Structural Modeling (Use Cases)
Comparing Modeling Styles
Summary of Key Takeaways
Example: AND Gate (All 3 styles)
Example: Half Adder (All 3 styles)
What is a Verilog Testbench?
Testbench Example: AND Gate

Description
What is Verilog and why it's important
Modules, ports, data types
Anatomy of a Verilog module
Top-level and submodules
Overview of 3 styles
always blocks and reg
FSMs, control logic
assign and expressions
Arithmetic and logic
Gate-level design
Post-synthesis netlists
Abstraction, usage, syntax
When to use which style
Behavioral, Dataflow, Structural
Behavioral, Dataflow, Structural
Purpose, structure, key constructs
Simulation-based verification in action

What is Verilog?

- HDL to model digital systems
- Used for simulation and synthesis
- Describes behavior, structure, and dataflow
- Essential for ASIC/FPGA design

Basic Verilog Syntax

- Case-sensitive, C-like syntax
- **Keywords:** module, endmodule, input, output, wire, reg, assign, always

Example:

```
module and_gate (input a, input b, output y);  
    assign y = a & b;  
endmodule
```


Verilog Module Structure

- module_name with ports
- Declare inputs/outputs, wires, regs
- Contain logic or instances

Example:

```
module my_module(input x, output y);  
  // logic  
endmodule
```

➔ Verilog Design Hierarchy

- Top-level modules can contain submodules
- Promotes reusability and manageability

Example:

```
module top();  
    wire a, b, y;  
    and_gate u1(a, b, y);  
endmodule
```


➔ Types of Modelling

- Each describes the same circuit differently.

Behavioural Modelling

Dataflow Modelling

Structural Modelling

➔ Behavioral Modeling – Overview

- Describes behavior using always blocks
- Uses 'reg' data type

Example:

```
module behavioral_and (input a, input b, output reg y);  
    always @(a or b)  
        y = a & b;  
endmodule
```

Behavioral Modeling – Use Cases

- Control logic (FSMs)
- Sequential circuits
- Testbenches
- Human-readable logic

Dataflow Modeling – Overview

- Uses continuous assignment (assign)
- Focus on data flow

Example:

```
module dataflow_and (input a, input b, output y);  
    assign y = a & b;  
endmodule
```

Dataflow Modeling – Use Cases

- Arithmetic/combinational logic
- Synthesizable
- Efficient for RTL coding

Structural Modeling – Overview

- Connects gates/modules
- Mirrors circuit schematic

Example:

```
module structural_and (input a, input b, output y);  
    and u1(y, a, b);  
endmodule
```

➔ Structural Modeling – Use Cases

- From schematic/netlist
- Gate-level simulation
- Used post-synthesis

Example:

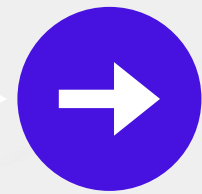
```
module structural_and (input a, input b, output y);  
    and u1(y, a, b);  
endmodule
```


→ Comparison of Modeling Styles

Feature	Behavioral	Dataflow	Structural
Abstraction level	High	Medium	Low
Uses	FSMs, sequences	Arithmetic/logic	Gate netlists
Syntax	always, if	assign	module, and
synthesizable	Yes(with care)	Yes	Yes

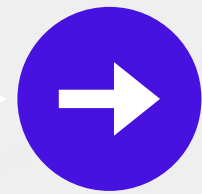
Summary

- Verilog supports multiple abstraction levels
- Choose style based on use-case
- All styles are synthesizable
- Useful in SoC, IP, and system design



Verilog Code Examples AND Gate in 3 Modeling Styles

Behavioral Modeling	Dataflow Modeling	Structural Modeling
<pre>module and_behavioral(inp ut a, input b, output reg y); always @(a or b) y = a & b; endmodule</pre>	<pre>module and_dataflow(input a, input b, output y); assign y = a & b; endmodule</pre>	<pre>module and_structural(input a, input b, output y); and u1(y, a, b); endmodule</pre>



Verilog Code Examples – Half Adder in 3 Modeling Styles

Behavioral Modeling	Dataflow Modeling	Structural Modeling
<pre> module half_adder_behavioral (input a, input b, output reg sum, output reg carry); always @(a or b) begin sum = a ^ b; carry = a & b; end endmodule </pre>	<pre> module half_adder_dataflow (input a, input b, output sum, output carry); assign sum = a ^ b; assign carry = a & b; endmodule </pre>	<pre> module half_adder_structural (input a, input b, output sum, output carry); xor u1(sum, a, b); and u2(carry, a, b); endmodule </pre>

What is a Verilog Testbench?

- **Definition:**
 - A testbench is a Verilog module used for simulation only.
 - It provides inputs and checks outputs of the DUT (Device Under Test).
- **Key Features:**
 - No input/output ports
 - Uses 'initial', '#delay', '\$display', '\$finish'
 - Instantiates the DUT

➔ Testbench for **AND Gate**

```
module tb_and_gate;
  reg a, b;
  wire y;

  and_dataflow dut (.a(a), .b(b), .y(y));

  initial begin
    $display("Time\t a b | y");
    $monitor("%0t\t %b %b | %b", $time, a, b, y);
    a = 0; b = 0;
    #10 a = 0; b = 1;
    #10 a = 1; b = 0;
    #10 a = 1; b = 1;
    #10 $finish;
  end
endmodule
```

Thank You

echiphu

